



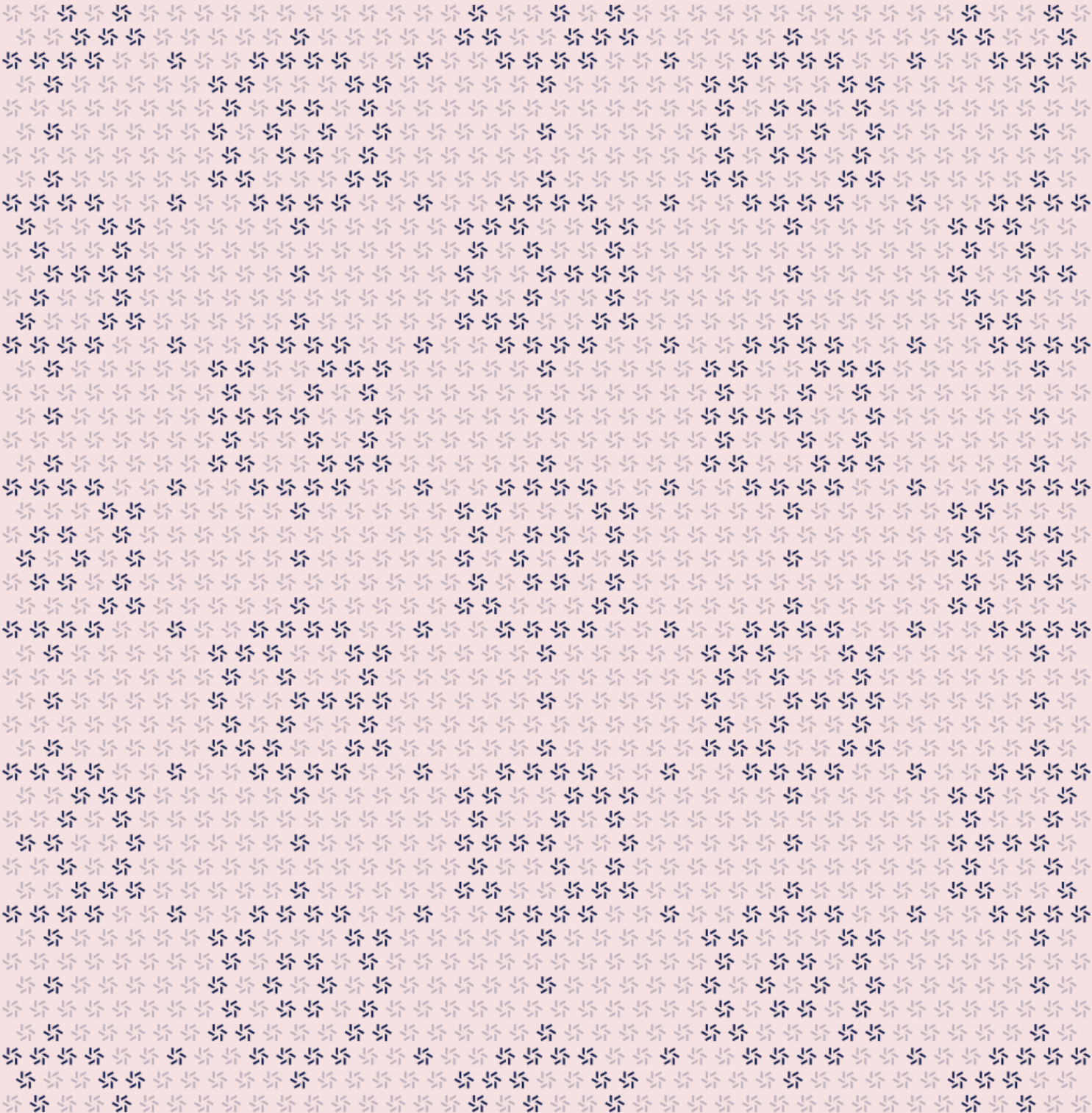
Prepared for
Ryan Zarick
Isaac Zhang
LayerZero Labs

Prepared by
Katerina Belotskaia
Weipeng Lai
Daniel Lu
Nan Wang
Zellic

June 26, 2025

USDT Native Mesh 1.1 Upgrade

Ton Application Security Assessment



Contents

About Zellic 4

1. Overview 4

- 1.1. Executive Summary 5
- 1.2. Goals of the Assessment 5
- 1.3. Non-goals and Limitations 5
- 1.4. Results 5

2. Introduction 6

- 2.1. About USDT Native Mesh 1.1 Upgrade 7
- 2.2. Methodology 7
- 2.3. Scope 9
- 2.4. Project Overview 10
- 2.5. Project Timeline 10

3. Detailed Findings 10

- 3.1. Inconsistent optionsLength variable definition 11
 - 3.2. Outdated comments in the TON contract 12
 - 3.3. Imprecise error handling in TON send0FT flow 14
 - 3.4. Inconsistent storage-access conventions in the TON contract 16
 - 3.5. Missing sliceRefs check in compose-message traversal 19
 - 3.6. The burnAndNilifyAdmin role is not initialized in the constructor 21
-

4.	Discussion	22
4.1.	Inconsistent error-code naming convention	23
5.	System Design	23
5.1.	Component: USDT OFT TON contract (version 1.1 upgrade)	24
5.2.	Component: USDT OFT EVM contract (version 1.1 upgrade)	25
5.3.	Component: MultiHopComposer EVM contract (introduced in version 1.1)	27
6.	Assessment Results	27
6.1.	Disclaimer	28

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](https://twitter.com/zellic_io) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for LayerZero Labs from June 9th to June 12th, 2025. During this engagement, Zellic reviewed USDT Native Mesh 1.1 Upgrade's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Can the compose-message byte-validation logic (`_assertComposeMessageBytes`) be bypassed, and is the `maxComposeMessageBytes` limit sufficient?
 - Does the USDT Native Mesh 1.1 upgrade correctly implement LZ compose-payload functionality, particularly regarding potential encoding errors?
 - Do the newly added or adjusted routing table functions in `handler.fc` have sufficient permission checks to prevent unauthorized calls?
 - After the `Credits` object refactoring and Solana chain integration, does the system maintain functional consistency with previous behavior?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

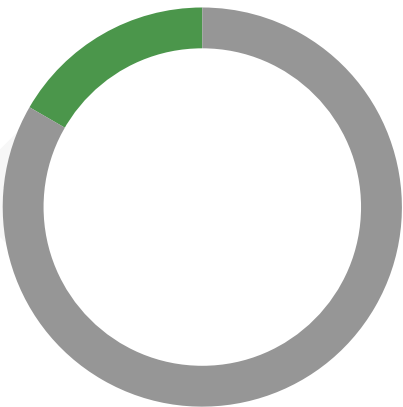
Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped USDT Native Mesh 1.1 Upgrade files, we discovered six findings. No critical issues were found. One finding was of low impact and the other findings were informational in nature.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	1
Informational	5



2. Introduction

2.1. About USDT Native Mesh 1.1 Upgrade

LayerZero Labs contributed the following description of USDT Native Mesh 1.1 Upgrade:

LayerZero is a technology that enables applications to move data across blockchains, uniquely supporting censorship-resistant messages and permissionless development through immutable smart contracts.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the files.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood.

We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

DRAFT

2.3. Scope

The engagement involved a review of the following targets:

USDT Native Mesh 1.1 Upgrade Files

Types	FunC, Solidity
Platforms	TON, EVM
Target	Only changes between c3c46856...ad758062
Repository	https://github.com/LayerZero-Labs/x-usdtoft
Version	ad75806292bfb1596e2da77a6b9026f9e6cb0c3f
Programs	packages/usdtoft-ton/src/**/*.fc packages/usdtoft-ton/structs/**/*.fc
Target	Only changes between d0b73031...ad758062
Repository	https://github.com/LayerZero-Labs/x-usdtoft
Version	ad75806292bfb1596e2da77a6b9026f9e6cb0c3f
Programs	packages/usdtoft-evm/contracts/**/*.sol

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 2 person-weeks. The assessment was conducted by four consultants over the course of four calendar days.

Contact Information

The following project managers were associated with the engagement:

✈ **Jacob Goreski**
Engagement Manager
jacob@zellic.io ↗

✈ **Chad McDonald**
Engagement Manager
chad@zellic.io ↗

The following consultants were engaged to conduct the assessment:

✈ **Katerina Belotskaia**
Engineer
kate@zellic.io ↗

✈ **Weipeng Lai**
Engineer
weipeng.lai@zellic.io ↗

✈ **Daniel Lu**
Engineer
daniel@zellic.io ↗

✈ **Nan Wang**
Engineer
nan@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

June 9, 2025 Start of primary review period

June 12, 2025 End of primary review period

3. Detailed Findings

3.1. Inconsistent optionsLength variable definition

Target	packages/usdtoft-ton/src/usdtOFT/coders.fc		
Category	Coding Mistakes	Severity	Low
Likelihood	Medium	Impact	Low

Description

In the coders.fc file within the `_encodeOfftSendAsComposeMessage` function, the `optionsLength` variable is defined twice with different values:

```
int optionsLength = 4 * 256 / 8; // 128 bytes (0x80)
...
int optionsLength = 0x50; // 80 bytes
```

The first definition calculates `optionsLength` as 128 bytes ($4 * 256 / 8 = 128 = 0x80$), while the second definition sets it to 80 bytes ($0x50$).

Impact

This inconsistency creates ambiguity about the intended `optionsLength` value and may lead to incorrect message encoding, potentially causing cross-chain communication failures or gas-estimation issues depending on which value is used in the final encoding.

Recommendations

Remove the duplicate `optionsLength` definition and determine the correct value based on the intended message structure.

Remediation

This issue has been acknowledged by LayerZero Labs, and a fix was implemented in commit [ecc5fc8e](#).

3.2. Outdated comments in the TON contract

Target	packages/usdtoft-ton/src/usdtOFT/handlerUsdt.fc		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

A much older version of the TON contract implementation included a function that looked up arbitrary fields in storage and modified them as if they were type `coin`. For example, we see the following snippet in commit [b8d7ee61a](#).

```
;; Very important to note this can only be applied to fields of type 'coin'
cell _incrementCoins(cell $storage, int field, int amount) impure inline {
    int currentAmount = $storage.cl::get<coins>(field);
    return $storage.cl::set(field, currentAmount + amount);
}
```

The analogous function in the current implementation is very different and no longer looks up arbitrary fields in storage. In particular, the changes in this engagement's scope modify the UsdtOFT storage to include credits in a dedicated object. Now, at the time of writing, it is implemented as follows.

```
;; Very important to note this can only be applied to fields of type 'coin'
cell _addCredits(cell $storage, int eid, int amount) impure inline {
    cell $Credits = UsdtOFT::getCredits($storage);
    cell $newCredits = Credits::increment($Credits, _getCreditsField(eid),
    amount);
    return $storage.UsdtOFT::setCredits($newCredits);
}
```

There is no longer a reason to keep this old comment, as the `Credits` object only has fields of type `coin`.

Impact

This can cause confusion for developers reading or maintaining the code, as it suggests a risk that no longer exists.

Recommendations

We recommend updating the comment to reflect the current implementation and remove similar references to the old functionality that no longer applies.

Remediation

TBD

DRAFT

3.3. Imprecise error handling in TON sendOFT flow

Target	packages/usdtoft-ton/src/usdtOFT/handlerUsdt.fc		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The TON contract often branches on the destination eID to determine what state to modify and what parameters to retrieve. Generally, these include assertions that the provided eID falls within the space of valid destinations.

```
int _getCreditsField(int eid) impure inline {
    if (eid == ARBITRUM_EID) {
        return Credits::arbitrum;
    } elseif (eid == CELO_EID) {
        return Credits::celo;
    } elseif (eid == ETH_EID) {
        return Credits::eth;
    } elseif (eid == SOLANA_EID) {
        return Credits::solana;
    } elseif (eid == TON_EID) {
        return Credits::ton;
    } elseif (eid == TRON_EID) {
        return Credits::tron;
    } else {
        throw(ERROR::InvalidEid);
    }

    ;; This will never be reached, but compiler cant read
    return 0;
}
```

We notice that `_assertOptionsCost` does this not by explicitly handling the final case but by causing an implicit error later down the line.

```
() _assertOptionsCost(cell $enforcedOptions, cell $extraOptions, int dstEid,
int layerZeroFee) impure inline {
    cell $CostAsserts = getContractStorage().UsdtOFT::getCostAsserts();

    ;; If invalid EID, ensure an error will be thrown by integer overflow
```

```
int maxPriceRatio = MAX_U256;
int maxGasPrice = MAX_U256;

if (dstEid == ARBITRUM_EID) {
    (maxPriceRatio, maxGasPrice) =
$CostAsserts.CostAsserts::getArbitrum();
} elseif (dstEid == CELO_EID) {
    (maxPriceRatio, maxGasPrice) = $CostAsserts.CostAsserts::getCelo();
} elseif (dstEid == ETH_EID) {
    (maxPriceRatio, maxGasPrice) = $CostAsserts.CostAsserts::getEth();
} elseif (dstEid == SOLANA_EID) {
    (maxPriceRatio, maxGasPrice) = $CostAsserts.CostAsserts::getSolana();
} elseif (dstEid == TRON_EID) {
    (maxPriceRatio, maxGasPrice) = $CostAsserts.CostAsserts::getTron();
}
```

Impact

This may create confusing error messages for developers and users.

Recommendations

We recommend explicitly handling the case where an invalid eID is provided.

Remediation

TBD

3.4. Inconsistent storage-access conventions in the TON contract

Target	packages/usdtoft-ton/src/usdtOFT/handlerUsdt.fc		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

Like other LayerZero Labs contracts, the UsdtOFT contract has a state object formatted in their funC++ convention. However, it includes some fast paths for field accesses that avoid some dynamic offset calculations. So getting and setting fields is usually done through explicitly written getters and setters instead of the generic `classlib` functions on fields.

```
cell UsdtOFT::getOAppStorage(cell $UsdtOFT) impure inline {
    return $UsdtOFT.cellPreloadRefAt(0);
}

cell UsdtOFT::getCredits(cell $UsdtOFT) impure inline {
    return $UsdtOFT.cellPreloadRefAt(1);
}

int UsdtOFT::getContractBalance(cell $UsdtOFT) impure inline {
    return $UsdtOFT.cellPreloadCoinsAt(UsdtOFT::_contractBalanceOffset);
}
```

There are a number of these functions written that still fall back to the standard `classlib` accessors, which theoretically will eventually be replaced by more efficient versions.

```
;; todo: serde for this
int UsdtOFT::getMaxComposeMsgBytes(cell $UsdtOFT) impure inline {
    return $UsdtOFT.cl::get<uint32>(UsdtOFT::maxComposeMessageBytes);
}

;; ===== Object Setters =====

;; todo: implement an optimized version / serde of these things
cell UsdtOFT::setCredits(cell $UsdtOFT, cell $credits) impure inline {
    return $UsdtOFT.cl::set(UsdtOFT::credits, $credits);
}

cell UsdtOFT::setContractBalance(cell $UsdtOFT, int amount) impure inline {
```

```

        return $UsdtOFT.cl::set(UsdtOFT::contractBalance, amount);
    }

    cell UsdtOFT::setFeeBalance(cell $UsdtOFT, int amount) impure inline {
        return $UsdtOFT.cl::set(UsdtOFT::feeBalance, amount);
    }

```

However, despite these functions being written, some of the code still uses the underlying `cl::get` and `cl::set` functions directly. There is also no setter for `maxComposeMessageBytes` written for `UsdtOFT`.

```

tuple setFee(cell $Fee) impure inline {
    (cell $storage, tuple actions) = preamble();
    int bps = $Fee.Fee::getBps();

    ;; fees cant be 100%
    throw_unless(ERROR::InvalidFee, bps < BPS_DENOMINATOR);

    setContractStorage($storage.cl::set(UsdtOFT::feeBps, bps));
    actions~pushAction<event>(EVENT::FeeSet, $Fee);

    return actions;
}

tuple setMaxComposeMsgBytes(cell $MsgSize) impure inline {
    (cell $storage, tuple actions) = preamble();
    int size = $MsgSize.MsgSize::getSize();

    setContractStorage($storage.cl::set(UsdtOFT::maxComposeMessageBytes,
    size));
    actions~pushAction<event>(EVENT::MaxComposeMsgBytesSet, $MsgSize);

    return actions;
}

```

Impact

This inconsistency means that if the `UsdtOFT` storage functions are ever optimized, some of the code will not benefit from the optimizations without further changes to actually use the optimized functions.

Recommendations

We recommend updating the code to consistently use the explicitly defined getters and setters for all fields in the UsdtOFT contract.

Remediation

TBD

DRAFT

3.5. Missing sliceRefs check in compose-message traversal

Target	packages/usdtoft-ton/src/usdtOFT/coders.fc		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

In the compose-message handling logic (e.g., `_assertComposeMessageBytes`), the code traverses chained cells by repeatedly taking the first reference (`preload_first_ref()`) and parsing it. However, there is currently no explicit check that each cell in the chain has exactly one reference (i.e., `sliceRefs == 1`).

Without this check, a malformed cell could have multiple references, which would be silently ignored by the traversal logic.

Impact

If a cell in the compose-message chain contains more than one reference, only the first is traversed and the rest are skipped. This could lead to

- malformed or ambiguous message structures,
- the potential for hidden payloads or data that bypass validation, and
- an increased risk of downstream parsing errors or security issues.

Recommendations

Add an explicit check to ensure that each cell in the compose-message chain has exactly one reference during traversal:

```
while (sliceRefs > 0) {
    throw_unless(ERROR::InvalidComposePayload, sliceRefs == 1);
    messageSlice = messageSlice.preload_first_ref().begin_parse();
    // ...
}
```

Remediation

This issue has been acknowledged by LayerZero Labs, and a fix was implemented in commit [ecc5fc8e](#).

DRAFT

3.6. The burnAndNilifyAdmin role is not initialized in the constructor

Target	packages/usdtoft-evm/contracts/UsdtOFT.sol		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The EVM UsdtOFT contract introduces a new burnAndNilifyAdmin role but does not initialize it in the constructor, unlike other administrative roles. While lpAdmin and planner are both initialized to _delegate in the constructor, burnAndNilifyAdmin remains uninitialized:

```
// Management addresses
address public lpAdmin;
address public planner;
address public burnAndNilifyAdmin; // Not initialized in constructor

constructor(
    // ... parameters
    address _delegate
) OApp(_lzEndpoint, _delegate) Ownable(_delegate) {
    // ... other initialization
    lpAdmin = _delegate; // Initialized
    planner = _delegate; // Initialized
    // burnAndNilifyAdmin is not initialized, remains address(0)
}
```

This creates an inconsistency in role initialization patterns and requires the owner to manually call setBurnAndNilifyAdmin() after deployment to enable the burn and nilify functionality.

Impact

This initialization inconsistency could lead to operational issues where the burn and nilify functions remain inaccessible until manual initialization, potentially disrupting intended deployment workflows and creating a window where critical emergency functions are unavailable.

Recommendations

Initialize burnAndNilifyAdmin in the constructor, consistent with other administrative roles to ensure all functionality is available immediately after deployment.

```
constructor(  
    // ... parameters  
    address _delegate  
) OApp(_lzEndpoint, _delegate) Ownable(_delegate) {  
    // ... other initialization  
    lpAdmin = _delegate;  
    planner = _delegate;  
    burnAndNilifyAdmin = _delegate; // Add this initialization  
}
```

Remediation

TBD

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment.

4.1. Inconsistent error-code naming convention

The TON contract contains an inconsistent error-code naming convention in `interface.fc`. While most error codes follow the proper double-colon namespace separator format (`ERROR::`), one error code is missing a colon:

```
const int ERROR::InvalidLzConfigOpCode = "ERROR::InvalidLzConfigOpCode" &
    ERRORCODE_MASK;
const int ERROR:UnknownOptions = "ERROR::UnknownOptions" & ERRORCODE_MASK;
// Missing one colon
```

The `ERROR:UnknownOptions` should be `ERROR::UnknownOptions` to maintain consistency with the established naming convention.

Remediation

This issue has been acknowledged by LayerZero Labs, and a fix was implemented in commit [ecc5fc8e](#).

5. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

5.1. Component: USDT OFT TON contract (version 1.1 upgrade)

Description

The TON USDT OFT contract serves as the TON blockchain implementation of the LayerZero Omnichain Fungible Token (OFT) protocol, enabling cross-chain USDT transfers between TON and other supported chains (Arbitrum, Celo, ETH, Solana, TRON).

Through a message-routing system in `handler.fc` that processes different operation codes, a unified `Credits` object is used for multichain liquidity tracking, compose-message validation, and encoding in `coders.fc`.

Notable changes

The following outlines the notable changes in version 1.1.

1. The `Credits` object restructuring

- Migrated from scattered `uint128` fields to a unified `structs/Credits.fc` object
- Consolidated six chains (Arbitrum, Celo, ETH, Solana, TON, and TRON) into a single `objRef`
- Updated all APIs to use `getCredits()` / `setCredits()` with `Credits::increment` / `Credits::decrement`

2. Solana integration

- Added `SOLANA_EID` constants
- Extended `CostAsserts` with `maxPriceRatioSolana` / `maxGasPriceSolana`
- Updated credit field mappings and gas-price validation for Solana

3. Compose-message support

- Added new `coders.fc` with `_encodeSendOFT(msgType, to, amount, composeMessage)`
- Added strict validation via `_assertComposeMessageBytes` and `_validateAndEncodeComposeMessage`
- Added `maxComposeMessageBytes` storage limit and `sendOFTComposeGas` assertion
- Added new opcodes — `SetMaxComposeMsgBytes` with corresponding events

4. Interface and operational expansions

- Added new error codes: `InvalidComposePayload`, `InsufficientCreditsSolana`, `UnknownOptions`
- Updated permission routing in `handler.fc` for `WithdrawFees` (now `lpAdmin` is accessible)
- Added `MsgSize.fc` struct for parameter handling

5. Gas/cost assertion refactoring

- Expanded `GasAsserts::New()` from five to six parameters (added `sendOFTComposeGas`)
- Extended `CostAsserts` to 10 fields, including Solana support
- Renamed getters to return tuples: `getArbitrum()` to `(maxPriceRatio, maxGasPrice)`

Invariants

- **Compose-message size limits.** All compose messages must not exceed `maxComposeMessageBytes` to prevent cell overflow.
- **Address validation.** Compose messages must have valid sender addresses (either the original sender or `NULLADDRESS` for replacement).
- **Gas bounds.** All operations must respect per-chain gas-price limits defined in `CostAsserts`.
- **Permission boundaries.** Only authorized roles (`owner`, `lpAdmin`, `planner`) can perform privileged operations according to the routing table.

Attack surface

- Compose-message payloads through cross-chain send operations — these are validated by `_assertComposeMessageBytes` for size limits and `_validateAndEncodeComposeMessage` for address consistency.
- Gas parameters in message options — these are constrained by `CostAsserts` per-chain limits and `sendOFTComposeGas` bounds.
- Cross-chain message amounts — these are checked against per-chain credit limits and total supply constraints.
- Operational parameters via governance calls — these are protected by `owner/lpAdmin/planner` role checks in handler routing.

5.2. Component: USDT OFT EVM contract (version 1.1 upgrade)

Description

The `UsdtOFT` contract enables cross-chain USDT transfers among the chains that form the USDT Native Mesh.

Notable changes

The following outlines the notable changes in version 1.1.

1. Compose-message support

- Added a new `SEND_OFT_AND_CALL` message type
- Updated `_buildMsgAndOptions` to process the user's `_sendParam.composeMsg` input
- Introduced an execution path in `_lzReceive` to handle `SEND_OFT_AND_CALL` messages

2. Burn and nilify methods' wrapper

- Added a `nilify` wrapper function to call the endpoint's `nilify` function
- Added a `burn` wrapper function to call the endpoint's `burn` function
- Assigned the `nilify` and `burn` permissions to a special role, `burnAndNilifyAdmin`
- Added a `setBurnAndNilifyAdmin` function, allowing the owner to assign the `burnAndNilifyAdmin` role

3. Fee-withdrawal permissions update

- Granted the `LpAdmin` role permission to withdraw fees, in addition to the owner, through the `withdrawFees` function

4. Solana integration preparation

- Added the immutable variable `SOLANA_EID`
- Implemented credits accounting for Solana in `sendCredits` and `_receiveCredits`

Invariants

- **Amount limits.** The amount a user sends in `send` must not exceed the credits available on the destination chain (`credits[dstEid]`). The credits transferred in `sendCredits` must not exceed the credits available on the destination chain (`credits[dstEid]`). Additionally, the withdrawal amount in `withdrawRemote` must not exceed the credits available on the destination chain (`credits[dstEid]`), and the withdrawal amount in `withdrawLocal` must not exceed the credits available on the local chain (`credits[LOCAL_EID]`).
- **Permission boundaries.** Only authorized roles (`owner`, `lpAdmin`, `planner`, `burnAndNilifyAdmin`) can perform privileged operations, as enforced by access-control modifiers.

Attack surface

- Transfer amount in `send` — this is checked against the credit limit and restricted by the user's approved amount and existing balance.
- Composed message in `send` — `UsdtOFT` encodes the `addressToBytes32(msg.sender)` as the composed-message caller on the source chain. The composed message is

passed onto the Composer contract and processed in its `lzCompose` function.

5.3. Component: MultiHopComposer EVM contract (introduced in version 1.1)

Description

The MultiHopComposer contract enables cross-mesh transfers between the Adapter Mesh and the OFT Mesh. It receives composed messages, decodes them, and forwards them to the appropriate OFT contract. If a message fails to decode or send, it records the failure and allows anyone to either retry the transfer or refund the tokens back to the source chain.

Invariants

- **Failed-message management.** The amount of tokens forwarded in a retry or refund is always set to the amount originally transferred into the composer, preventing manipulation of locked funds. Failed messages are only deleted from storage after a successful refund or retry, preventing double-spending.
- **Permission boundaries.** The `lzCompose` can only be called by the LayerZero endpoint and can only receive composed messages from either Adapter Mesh or OFT Mesh.

Attack surface

- The `_extraOptions` parameter in `refund` and `retry` is user-supplied and passed to the OFT `send` call. This is checked by the OFT contract.
- The `refund` and `retry` functions are permissionless but can only be called for messages that have failed and are stored in `failedMessages`. The contract ensures that only valid failed messages can be retried or refunded.

6. Assessment Results

During our assessment on the scoped USDT Native Mesh 1.1 Upgrade files, we discovered six findings. No critical issues were found. One finding was of low impact and the other findings were informational in nature.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.